Dal web al desktop: Electron e Tauri

Corso di Frontend Development – Modulo 02

Federico Giuggioloni

ITS – Corso di Frontend Development

Anno formativo 2026/2027

Contenuto del modulo

1. Dal web al desktop: Electron e Tauri
1.1 Perché un'app desktop
1.2 Electron
1.3 Tauri
1.4 Confronto e conclusioni

Sommario del modulo

1. Dal web al desktop: Electron e Tauri
1.1 Perché un'app desktop
1.2 Electron
1.3 Tauri
1.4 Confronto e conclusioni

Il caso d'uso: il PC del negozio

Chi usa l'area admin non è un cliente: è chi sta dietro il bancone, tutto il giorno, sullo stesso computer.

- Icona sul desktop e avvio all'accensione.
- **Icona nella barra di sistema** con il numero di articoli esauriti.
- Notifiche native quando una scorta scende sotto la soglia.
- Scorciatoia globale (`Ctrl+Shift+N`: nuovo articolo) anche a finestra ridotta.
- **Esportare il catalogo** in CSV, con una vera finestra di salvataggio.
- Stampare le etichette dei prezzi.
- Lettore di codici a barre USB: si comporta da tastiera, ma serve una finestra con il fuoco.
- Nessuna barra degli indirizzi: non si naviga altrove e non si chiude la scheda per sbaglio.

Il patto

Riusiamo **tutto** il codice React del modulo 1: aggiungiamo un "guscio" nativo e un canale per parlarci.

Le due strade

Electron (2013, GitHub)

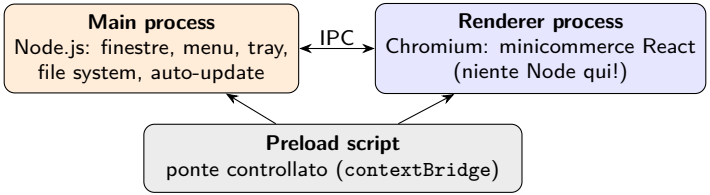
- Impacchetta **Chromium + Node.js** con l'app.
- Stesso browser ovunque: ciò che funziona su Windows funziona su Linux.
- VS Code, Slack, Discord, Figma, Obsidian, 1Password.
- Costo: 150–250 MB di installazione, memoria da browser.

Tauri (2020, v2 dal 2024)

- Usa la **webview di sistema** + un backend in **Rust**.
- Eseguibili da 5–15 MB, meno memoria, avvio rapido.
- Webview diverse per OS: piccole differenze da testare.
- Modello di sicurezza a permessi espliciti; v2 gira anche su Android/iOS.

Le facciamo entrambe: Electron perché lo troverete nelle aziende, Tauri perché è la direzione attuale. Il laboratorio principale è su Tauri.

Architettura di Electron



- **Un** main process, **un** renderer per finestra. Parlano solo via **IPC** (messaggi).
- Il renderer è una pagina web: se avesse Node, un XSS nella descrizione di un articolo diventerebbe un accesso completo al PC del negozio.
- Il preload espone al renderer *solo* le funzioni che decidiamo noi.

Setup: una cartella, nessun generatore

```
# dalla radice del repository, una volta sola
node scripts/setup-electron.mjs
```

```
desktop/                    # il guscio nativo, accanto a frontend/ e backend/
  package.json              # electron, electron-builder, esbuild
  src/main.ts               # processo principale
  src/preload.ts            # il ponte
  scripts/dev.mjs           # esbuild -> Vite -> Electron, in quest'ordine
  electron-builder.yml
frontend/                   # NON duplicato: resta uno solo
```

- Esistono generatori (`electron-vite`, Electron Forge) che creano un progetto nuovo in cui poi *copiare* il frontend. Qui no: il frontend resta uno solo e il guscio gli sta accanto.
- Due file di TypeScript compilati da **esbuild** in un comando: niente da configurare, e main e preload si leggono per intero.
- Lo script fa tutto: dipendenze, file del guscio, patch al frontend. Lo vediamo passo per passo nelle prossime slide.

Il main process: creare la finestra

```
import { app, BrowserWindow, shell } from 'electron';
import { join } from 'node:path';

function createWindow() {
  const win = new BrowserWindow({
    width: 1280, height: 800, minWidth: 1024,
    show: false,
    webPreferences: {
      preload: join(__dirname, '../preload/index.js'),
      contextIsolation: true, nodeIntegration: false, sandbox: true,  // il renderer NON vede Node
    },
  });
  win.once('ready-to-show', () => win.show());   // niente flash bianco
  win.webContents.setWindowOpenHandler(({ url }) => { shell.openExternal(url); return { action:
    'deny' }; });
  if (process.env.ELECTRON_RENDERER_URL) win.loadURL(process.env.ELECTRON_RENDERER_URL);  // dev
  else win.loadFile(join(__dirname, '../renderer/index.html'));                           // prod
}
app.whenReady().then(createWindow);
app.on('window-all-closed', () => { if (process.platform !== 'darwin') app.quit(); });
```

Le tre sorprese

1. **Il routing**: in produzione la pagina è caricata da `file://.../index.html`. Un `BrowserRouter` che naviga su `/admin` cerca *un file* che non esiste.
2. **L'origine**: con `file://` il browser invia `Origin: null` e il proxy di Vite non c'è più. L'API va chiamata al suo indirizzo assoluto, e il backend deve accettare quell'origine.
3. **Le immagini**: i data URL base64 degli articoli funzionano anche qui; percorsi relativi come `/uploads/mouse.png` no.

```
// nella SPA: scegliere il router in base all'ambiente
const createRouter = import.meta.env.VITE_ROUTER === 'hash' ? createHashRouter :
    createBrowserRouter;
// e nel .env del progetto desktop:
// VITE_ROUTER=hash
// VITE_API_URL=http://localhost:3001/api
```

In alternativa all'hash router: registrare un protocollo personalizzato (`app://`) che serve sempre `index.html` e dà un'origine stabile.

Preload e contextBridge: il ponte sicuro

```ts
// src/preload/index.ts
import { contextBridge, ipcRenderer } from 'electron';

const api = {
  notify: (title: string, body: string) => ipcRenderer.invoke('notify', { title, body }),
  exportCatalogo: (csv: string) => ipcRenderer.invoke('export-csv', csv),
  setEsauriti: (count: number) => ipcRenderer.send('badge', count),
  onNuovoArticolo: (cb: () => void) => { ipcRenderer.on('shortcut:nuovo-articolo', cb); },
  platform: process.platform,
};
contextBridge.exposeInMainWorld('minicommerce', api);
export type MinicommerceApi = typeof api;
```

```ts
// src/renderer/src/electron.d.ts
declare global { interface Window { minicommerce?: import('../../preload').MinicommerceApi } }
```

`window.minicommerce` è *opzionale*: lo stesso codice React gira anche nel browser, dove non esiste.

Main: gestire le richieste IPC

```ts
import { ipcMain, Notification, dialog, app, BrowserWindow } from 'electron';
import { writeFile } from 'node:fs/promises';

ipcMain.handle('notify', (_e, { title, body }) => { new Notification({ title, body }).show(); });

ipcMain.handle('export-csv', async (e, csv: string) => {
  const win = BrowserWindow.fromWebContents(e.sender)!;
  const { filePath, canceled } = await dialog.showSaveDialog(win, {
    defaultPath: `catalogo-${new Date().toISOString().slice(0, 10)}.csv`,
    filters: [{ name: 'CSV', extensions: ['csv'] }],
  });
  if (canceled || !filePath) return false;
  const BOM = String.fromCharCode(0xFEFF);          // Excel apre bene gli accenti
  await writeFile(filePath, BOM + csv, 'utf8');
  return true;
});

ipcMain.on('badge', (_e, count: number) => app.setBadgeCount(count));
```

Il main process è Node: tutto ciò che arriva dal renderer è **input non fidato** e va validato qui.

Dal lato React: usare il ponte se c'è

```
// hooks/usePiattaforma.ts
export function usePiattaforma() {
  const bridge = window.minicommerce;
  return {
    isDesktop: !!bridge,
    notify: (t: string, b: string) => (bridge ? bridge.notify(t, b) : notificaWeb(t, b)),
    exportCatalogo: (csv: string) =>
      bridge ? bridge.exportCatalogo(csv) : scaricaBlob(csv, 'catalogo.csv'),
  };
}

// AdminPage.tsx
const { exportCatalogo } = usePiattaforma();
const { data: products = [] } = useProducts({});
<Button variant="outline" onClick={() => exportCatalogo(toCsv(products))}>
  <Download className="size-4" /> Esporta catalogo
</Button>
```

Il fallback web (`scaricaBlob` crea un `<a download>`) mantiene **un solo codice** per browser e desktop.

Tray con il numero di esauriti

```
import { Tray, Menu, nativeImage } from 'electron';

tray = new Tray(nativeImage.createFromPath(join(__dirname, '../../resources/tray.png')));
tray.setContextMenu(Menu.buildFromTemplate([
  { label: 'Apri minicommerce', click: () => win.show() },
  { label: 'Nuovo articolo', accelerator: 'CmdOrCtrl+Shift+N',
    click: () => win.webContents.send('shortcut:nuovo-articolo') },
  { type: 'separator' },
  { role: 'quit' },
]));
tray.setToolTip('minicommerce');

// chiudere = nascondere nella tray, non uscire
win.on('close', (e) => { if (!app.isQuitting) { e.preventDefault(); win.hide(); } });
```

```
// nel renderer: a ogni ricarica dell'elenco articoli
useEffect(() => {
  window.minicommerce?.setEsauriti(products.filter(p => p.stock === 0).length);
}, [products]);
```

Il token: dove finisce su desktop

```
import { safeStorage } from 'electron';
import { readFile, writeFile } from 'node:fs/promises';
const file = join(app.getPath('userData'), 'session.bin');

ipcMain.handle('session:save', async (_e, token: string) => {
  await writeFile(file, safeStorage.encryptString(token));   // cifrato con il keychain dell'OS
});
ipcMain.handle('session:load', async () => {
  try { return safeStorage.decryptString(await readFile(file)); } catch { return null; }
});
```

- `localStorage` funziona anche in Electron, ma è un file in chiaro dentro il profilo utente. Sul PC condiviso del negozio non è il massimo.
- `safeStorage` usa DPAPI (Windows), Keychain (macOS), libsecret (Linux).
- Conseguenza sul codice: la lettura del token diventa **asincrona**, quindi l'interceptor di axios va reso `async` e l'`AuthProvider` parte da uno stato "sto caricando la sessione".

Pacchettizzare e distribuire

```
cd desktop
npm run package      # build del frontend + esbuild + electron-builder
                     # -> desktop/release/ (su Windows: NSIS, ~110 MB)
```

```
// electron-builder.yml (estratto)
appId: it.its.minicommerce
productName: minicommerce
win:   { target: ["nsis"] }
mac:   { target: ["dmg"], hardenedRuntime: true, notarize: true }
linux: { target: ["AppImage", "deb"], category: "Office" }
publish: { provider: "github" }
```

- **Firma del codice**: senza certificato Windows mostra "editore sconosciuto"; senza notarizzazione macOS blocca l'app.
- **Auto-update**: `electron-updater` scarica dalle release GitHub e aggiorna al riavvio. Per il negozio è essenziale: nessuno andrà a reinstallare a mano.

Checklist di sicurezza per Electron

- `contextIsolation: true`, `nodeIntegration: false`, `sandbox: true`.
- Esporre via `contextBridge` **funzioni specifiche**, mai `ipcRenderer` intero.
- Validare *nel main* i dati che arrivano dal renderer: il contenuto del CSV, il percorso, le dimensioni.
- `setWindowOpenHandler`: i link esterni si aprono nel browser, non in una finestra Electron senza barra.
- Content Security Policy nell'`index.html`, con attenzione ai data URL delle immagini degli articoli.
- Aggiornare Electron: ogni versione porta le patch di sicurezza di Chromium.

Riferimento: `https://www.electronjs.org/docs/latest/tutorial/security`

Laboratorio 2a

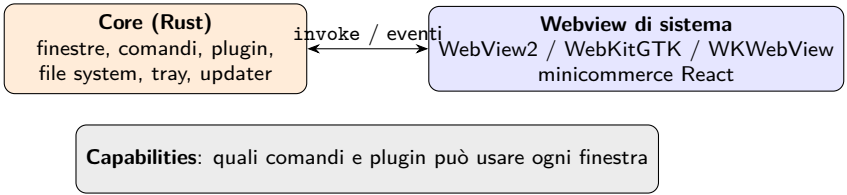
Laboratorio: minicommerce Desktop con Electron

1. Lanciare `node scripts/setup-electron.mjs` e far partire l'app: `cd desktop && npm run dev`.
2. Leggere `desktop/src/main.ts` e `preload.ts` e spiegare a voce ogni opzione di sicurezza.
3. Aggiungere al ponte una funzione vostra: scorciatoia globale "nuovo articolo" oppure contatore degli esauriti nella tray.
4. `npm run package` e installare il risultato. Confrontare il vostro lavoro con il branch `desktop/electron`.

Domande di verifica

Perché il renderer non deve avere accesso a Node? Che differenza c'è fra `ipcRenderer.invoke` e `send`? Perché `BrowserRouter` non funziona con `file://`?

Architettura di Tauri 2



- Il backend del guscio è un programma Rust: sicuro per costruzione, veloce, senza Node.
- Non conoscete Rust? Per il 90% dei casi bastano i **plugin ufficiali** e comandi di cinque righe.
- Le webview non sono Chromium: su Linux (WebKitGTK) qualche funzione CSS recente può mancare. Tailwind v4 usa CSS moderno: provate presto su Linux.

Setup sul frontend esistente

```
# prerequisiti: Rust (rustup); su Windows i Build Tools C++ e WebView2 (gia' in Win 11)
# su Linux: libwebkit2gtk-4.1-dev, libssl-dev, ...

# dalla radice del repository, una volta sola
node scripts/setup-tauri.mjs

# dentro, in sostanza, fa questo nel frontend:
npm install -D @tauri-apps/cli@^2 && npm install @tauri-apps/api@^2
npx tauri init --ci          # dev URL :5173, dist ../dist
npx tauri add dialog fs notification
npx tauri icon src/assets/hero.png
```

```
frontend/src-tauri/   Cargo.toml, tauri.conf.json, capabilities/, src/lib.rs, icons/
```

Notate la differenza con Electron: qui **non si crea un nuovo progetto**, si aggiunge una cartella a quello che avete già.

tauri.conf.json e capabilities

```
{
  "productName": "minicommerce",
  "identifier": "it.its.minicommerce",
  "build": { "devUrl": "http://localhost:5173", "frontendDist": "../dist",
             "beforeDevCommand": "npm run dev", "beforeBuildCommand": "npm run build:desktop" },
  "app": { "windows": [{ "title": "minicommerce", "width": 1280, "height": 800 }],
           "security": { "csp": "default-src 'self'; img-src 'self' data:; connect-src http://localhost:3001" } },
  "bundle": { "active": true, "targets": "all", "icon": ["icons/icon.png"] }
}
```

```
// capabilities/default.json: cosa puo' fare la finestra principale
{ "identifier": "default", "windows": ["main"],
  "permissions": ["core:default", "notification:default", "dialog:default",
                  "fs:allow-write-text-file", "store:default", "global-shortcut:default"] }
```

In Tauri l'app **non può** fare nulla che non sia elencato qui. È il contrario di Electron, dove tutto è permesso finché non lo chiudete voi.

Comandi Rust e invoke

```rust
// src-tauri/src/lib.rs
#[tauri::command]
fn esporta_csv(path: String, csv: String) -> Result<(), String> {
    std::fs::write(&path, csv).map_err(|e| e.to_string())
}

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    tauri::Builder::default()
        .plugin(tauri_plugin_notification::init())
        .plugin(tauri_plugin_dialog::init())
        .invoke_handler(tauri::generate_handler![esporta_csv])
        .run(tauri::generate_context!())
        .expect("errore avvio minicommerce");
}
```

```javascript
import { invoke } from '@tauri-apps/api/core';
await invoke('esporta_csv', { path, csv });   // Promise<void>, errori come eccezioni
```

I plugin: quasi tutto senza scrivere Rust

```
npx tauri add dialog fs notification          # crate Rust + pacchetto npm + permesso
```

```
import { sendNotification, isPermissionGranted, requestPermission } from '@tauri-apps/plugin-notification';
import { save } from '@tauri-apps/plugin-dialog';
import { writeTextFile } from '@tauri-apps/plugin-fs';
import { register } from '@tauri-apps/plugin-global-shortcut';
import { Store } from '@tauri-apps/plugin-store';

export async function exportCatalogo(csv: string) {
  const path = await save({ defaultPath: 'catalogo.csv', filters: [{ name: 'CSV', extensions: ['csv'] }] });
  if (path) await writeTextFile(path, csv);
}
export async function avvisaScorte(nomi: string[]) {
  if (!(await isPermissionGranted())) await requestPermission();
  sendNotification({ title: 'Scorte in esaurimento', body: nomi.join(', ') });
}
await register('CommandOrControl+Shift+N', () => apriNuovoArticolo());
const store = await Store.load('session.json');    // per cifrare: plugin stronghold
```

Una sola interfaccia, tre implementazioni

```ts
// lib/piattaforma.ts
export const piattaforma =
  '__TAURI_INTERNALS__' in window ? 'tauri' :
  window.minicommerce              ? 'electron' :
  'web';

export interface Piattaforma {
  notify(title: string, body: string): Promise<void>;
  exportCatalogo(csv: string): Promise<void>;
  onNuovoArticolo(cb: () => void): () => void;
}

const impl: Piattaforma =
  piattaforma === 'tauri' ? tauriImpl : piattaforma === 'electron' ? electronImpl : webImpl;
export const usePiattaforma = () => impl;
```

- `CatalogPage`, `AdminPage` e il dialog non sanno dove stanno girando.
- È lo stesso schema che useremo per il mobile, dove si aggiungerà un quarto caso: Capacitor.

Tray, eventi, build

```rust
use tauri::{tray::TrayIconBuilder, menu::{Menu, MenuItem}, Manager};
// dentro .setup(|app| { ... })
let quit = MenuItem::with_id(app, "quit", "Esci", true, None::<&str>)?;
let menu = Menu::with_items(app, &[&quit])?;
TrayIconBuilder::new().menu(&menu).icon(app.default_window_icon().unwrap().clone())
    .on_menu_event(|app, e| if e.id() == "quit" { app.exit(0) })
    .build(app)?;
```

```
npm run desktop:build   # -> src-tauri/target/release/bundle/{msi,nsis,deb,appimage,dmg}
```

- Eventi dal core alla webview: `app.emit("scorte", 3)` e `listen('scorte', ...)` in TypeScript.
- Aggiornamenti: plugin `updater` con un manifest JSON firmato (chiave generata da `tauri signer`).
- Anche qui: firma su Windows e notarizzazione su macOS per una distribuzione seria.

Laboratorio 2b

Laboratorio: minicommerce Desktop con Tauri

1. Lanciare `node scripts/setup-tauri.mjs`; `cd frontend && npm run desktop:dev`.
2. Leggere `capabilities/default.json` e provare a usare un plugin *non* elencato: guardare l'errore.
3. Aggiungere un comando Rust vostro accanto a `conta_esauriti` e chiamarlo con `invoke`.
4. `npm run desktop:build` e confronto delle dimensioni con il pacchetto Electron.
5. Completare `lib/piattaforma.ts` con l'implementazione che manca.

Domande di verifica

Cosa succede se chiamo un plugin non elencato nelle capabilities? Perché l'app Tauri pesa 10 MB e quella Electron 200? Cosa non posso dare per scontato sulla webview di Linux?

Electron vs Tauri

	Electron	Tauri 2
Motore web	Chromium incluso	Webview dell'OS
Backend del guscio	Node.js	Rust (+ plugin)
Dimensione	150–250 MB	5–15 MB
Memoria	alta	media/bassa
Coerenza del rendering	identica ovunque	da testare per OS
Sicurezza	opt-in (checklist)	permessi espliciti
Ecosistema	enorme, maturo	in crescita rapida
Mobile	no	sì (Android/iOS)
Curva di apprendimento	solo JS	JS + un po' di Rust

- Nuovo progetto nel 2026: **Tauri**, salvo bisogno di moduli nativi Node o di un rendering identico garantito.
- Progetto esistente con team solo JavaScript e molte dipendenze Node: Electron resta ragionevole.

Cosa abbiamo cambiato nel frontend (e cosa no)

Cambiato

- Router: hash sul desktop
- `VITE_API_URL` assoluto
- Token in storage cifrato, lettura asincrona
- Un modulo `piattaforma.ts` con tre implementazioni
- Link esterni aperti fuori dall'app

Non toccato

- Tutte le pagine e i componenti
- Gli hook dei dati, il form, lo schema zod
- Tailwind, shadcn/ui, il tema
- `lib/api.ts` (a parte la base URL)
- Il backend

Lezione

Progettare **confini** (il client HTTP, lo storage, le capacità native) dietro interfacce è ciò che rende un'app portabile. Non è "roba da architetti": lo avete appena fatto in due laboratori.

I due branch di riferimento

```
git switch desktop/electron    # master, un commit di merge, i PDF delle slide
git switch desktop/tauri       # idem: il ramo aggiunge una cosa sola

git diff master..desktop/electron -- frontend/   # le patch condivise
git diff desktop/electron..desktop/tauri -- frontend/src/lib/piattaforma.ts
```

- Ogni branch aggiunge **una cosa sola**, in un commit di merge: il suo messaggio racconta tutto ciò che è stato fatto. Leggetelo, è la sintesi della lezione.
- Il secondo `diff` mette una accanto all'altra le due implementazioni della stessa interfaccia: stessa UI, due mondi nativi diversi.
- I due script sono idempotenti: potete rilanciarli senza rompere nulla.

Alternative da conoscere

- **Neutralinojs**, **Wails** (Go): stessa idea di Tauri con altri linguaggi nel guscio.
- **PWA installata** (prossimo modulo): per il PC del negozio può bastare, senza installer né store.
- **.NET MAUI Blazor Hybrid**, **Flutter desktop**: interfaccia non web, stesso codice su desktop e mobile.
- **Nativo puro** (WinUI, SwiftUI, GTK): modulo 5. Quando il feeling e le prestazioni contano più del riuso.